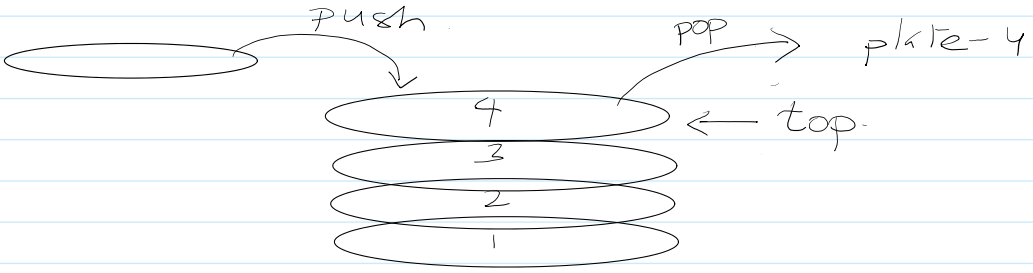
Stack Data Structure

08 August 2026 18:27

A Stack is a linear data structure that follows the LIFO(Last In, First Out) principle.
This means:
The element inserted last is the first element to remove.

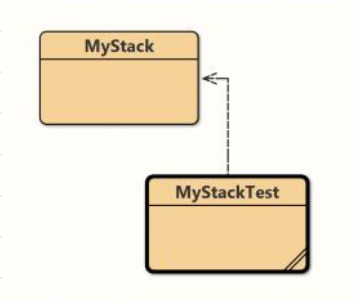
A real - world example is a stack of plates. If you place plate one above another, you can remove only the plate at the top.



Stack operations
A stack generally provides the following operations:

Operation	Meaning
push()	Insert an element
pop()	Remove the top element
peek()	View the top element without removing it.
isEmpty()	Check whether stack is empty
isFull()	Check whether stack is full
size()	Number of elements

Stack using Array:
We can implement a stack without using a Java's stack class by using a normal array.




```
package DataStructures.StackDataStructure;


/**
 * Write a description of class MyStack here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```
 */
public class MyStack
{
    private int stack[];
    private int top, count, capacity;

    public MyStack(int size){
        capacity = size;
        stack = new int[capacity];
        top = -1; //Because initially there are no elements in the stack
        count = 0;
    }

    public MyStack(){
        this(10);
    }

    public void push(int ele){
        if(isFull()){
            System.out.print("\nStack overflow!");
            return;
        }
        stack[++top] = ele;
        count++;
    }

    public boolean isFull(){
        return top == capacity - 1;
    }

    /**
     * @return number of elements present in the current MyStack object
     */
    public int numberOfElements(){
        return count;
    }

    public int pop(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }
        int v = stack[top--];
        count--;
        return v;
    }

    public int peek(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }

        return stack[top];
```

```
    }

    public boolean isEmpty(){
        return top == -1;
    }

    public int capacityOfStack(){
        return capacity;
    }
}


package DataStructures.StackDataStructure;
import java.util.*;

/**
 * Write a description of class MyProgram here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyStackTest
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        System.out.print("\f");
        int size;
        System.out.print("\nEnter the size of your stack: ");
        size = sc.nextInt();
        MyStack s = new MyStack(size);

        while(true){
            System.out.print("\n==============================");
            System.out.print("\n========Menu========");
            System.out.print("\n==============================");
            System.out.print("\n1. Push\n2. Pop\n3. Peek"+
                "\n4 Number of elements\n5 Capacity of Stack\n6 Exit"+
                "\nEnter your choice: ");
            int ch = sc.nextInt();
            switch(ch){
                case 1:
                    System.out.print("\nEnter integer value to push into Stack: ");
                    int e = sc.nextInt();
                    s.push(e);
                    break;
                case 2:
                    e = s.pop();
                    if(e == Integer.MIN_VALUE){
                        System.out.print("\nStack Underflow!");
                    }
                    else{
                        System.out.print("\nPopped element: " + e);
```

```
                    }
                    break;
                case 3:
                    e = s.peek();
                    if(e == Integer.MIN_VALUE){
                        System.out.print("\nStack is empty!");
                    }
                    else{
                        System.out.print("\nPeak element: " + e);
                    }
                    break;
                case 4:
                    System.out.print(
                        "\nNumber of elements present in the Stack: "+
                        s.numberOfElements());

                    break;
                case 5:
                    System.out.print("\nCapacity of stack: "+
                        s.capacityOfStack());
                    break;
                case 6:
                    System.out.print("\nGood bye");
                    System.exit(ch);
                default:
                    System.out.print("\nWrong menu item selected.");
            }
        }
    }
}
```

Output: Execute the code in your system.

Applications of Stack:

Stacks are widely used in computer science.

1. **Function Call:** When a method calls another method, Java uses a call stack to keep the track of method execution.

```
main()
  |
  V
method1()
  |
  V
method2()
```

When method2() finishes execution, returns to method1()

2. **Undo/Redo:** The text editors can maintain previous operations using Stack. Undo removes the most recent operation first.

3. **Browser History:** A stack like structure can be used to manage navigation history.

4. **Expression Evaluation:** Stacks are used to evaluate expressions such as 10 + 20 * 30

5. **Parentheses Matching:** A stack can check whether parentheses are balanced.
   [[()]]
   The expression is balanced
   For: {[(])} => This expression is not balanced.

6. **Depth-first Search (DFS):** DFS for graphs and trees can be implemented using a stack.

7. **Backtracking:** Stack structures are useful in problems such as:
   a) Maze solving
   b) Sudoku
   c) Path finding
   d) Backtracking algorithms

For linked list implementation:

1. LinkedList object
2. Push() => insertAtStart()
3. Pop() => deleteAtStart()
4. display()
5. checkEmpty()

Implement or write down a program using LinkedList class to implement linkedstack datastructure.

Practice Questions

Question 1: Menu-Driven Stack Operations Using an Array

Write a menu-driven Java program to implement a stack of integers using an array. The program should support all fundamental stack operations and demonstrate the LIFO (Last In, First Out) principle.

Requirements

1. Create a class named ArrayStack with the following data members:
   - int stack[] – to store elements.
   - int top – to track the topmost element.
   - int capacity – to store the maximum capacity of the stack.
2. Implement the following methods:
   - ArrayStack(int size) – initialize the stack with the specified capacity.
   - void push(int value) – insert an element into the stack.
   - int pop() – remove and return the topmost element.
   - int peek() – return the topmost element without removing it.
   - boolean isEmpty() – check whether the stack is empty.
   - boolean isFull() – check whether the stack is full.
   - int size() – return the number of elements currently present.
   - void display() – display all elements from top to bottom.
   - void search(int value) – search for an element in the stack and display its position.

3. Display a menu repeatedly until the user selects Exit.
4. Handle Stack Overflow when attempting to push into a full stack.
5. Handle Stack Underflow when attempting to pop or peek from an empty stack.
6. Validate the user's menu choices and prevent invalid operations.
7. Do not use built-in stack implementations or collection classes.

Sample output

```
Enter stack capacity: 5
========== STACK MENU ==========
1. Push
2. Pop
3. Peek
4. Display
5. Search
6. Size
7. Check Empty
8. Check Full
9. Exit
Enter choice: 1
Enter value: 10
10 pushed successfully.
Enter choice: 1
Enter value: 20
20 pushed successfully.
Enter choice: 1
Enter value: 30
30 pushed successfully.
Enter choice: 4
Stack elements (Top to Bottom):
30
20
10
Enter choice: 2
Popped element: 30
Enter choice: 3
Top element: 20
```

Additional challenges:

- Display the stack from bottom to top without modifying its final contents.
- Implement a method to return the sum and average of all stack elements.
- Count how many elements are even and how many are odd.

Question 2: Reverse a String and Check Palindromes Using a Stack

Write a Java program that uses a stack to reverse strings and determine whether a given string is a palindrome.

You must implement the stack manually using a char[] array. Do not use StringBuilder.reverse(), collection classes, or any built-in stack implementation.

Requirements

Create a class named CharStack with the following methods:

- void push(char ch) – push a character onto the stack.
- char pop() – remove and return the top character.
- boolean isEmpty() – check whether the stack is empty.
- boolean isFull() – check whether the stack is full.

- char peek() – inspect the top character.
- void display() – display the stack contents.

Then create a class named StringStackApplication that provides the following menu:
1. Reverse a string.
2. Check whether a string is a palindrome.
3. Count vowels and consonants.
4. Exit.

Detailed specifications

A. Reverse a string
- Accept a string from the user.
- Push each character onto the stack.
- Pop the characters one by one to construct the reversed string.
- Display both the original and reversed strings.

B. Check for a palindrome
- Accept a string such as MADAM, LEVEL, or NITIN.
- Ignore differences between uppercase and lowercase letters.
- Ignore spaces and punctuation.
- Use the stack to compare the processed string with its reverse.

For example, Never odd or even should be treated as a palindrome.

C. Count vowels and consonants
- Count vowels (a, e, i, o, u) without considering case.
- Count alphabetic consonants.
- Do not count spaces, digits, or punctuation.

D. Error handling
- Handle an empty input string.
- Ensure the stack capacity is sufficient for the processed string.
- Do not attempt to pop from an empty stack.

Sample output

```
========== STRING STACK MENU ==========
1. Reverse String
2. Check Palindrome
3. Count Vowels and Consonants
4. Exit
Enter choice: 1
Enter a string: JAVA PROGRAMMING
Original String : JAVA PROGRAMMING
Reversed String: GNIMMARGORP AVAJ
Enter choice: 2
Enter a string: Never odd or even
Processed String: neveroddoreven
The string is a palindrome.
```

Additional challenge: Extend the program to reverse the order of words in a sentence using a stack, without reversing the individual characters within each word.

Question 3: Balanced Parentheses and Expression Validation Using a Stack

Difficulty: Intermediate

Write a Java program to check whether mathematical expressions contain properly balanced parentheses, brackets, and braces using a stack implemented with an array. The program should recognize these pairs:
- (and)

- [and]
- { and }

Requirements

Create a class named BracketStack with the following methods:

- void push(char ch)
- char pop()
- char peek()
- boolean isEmpty()
- boolean isFull()

Create another class named ExpressionValidator that implements the following functionality.

Part A: Check balanced brackets

Accept an expression and process its characters from left to right.

1. When an opening bracket is encountered, push it onto the stack.
2. When a closing bracket is encountered, check whether the stack is empty.
3. If the stack is empty, the expression contains an unmatched closing bracket.
4. Otherwise, pop the top bracket and verify that it matches the closing bracket.
5. After processing all characters, the expression is balanced only if the stack is empty and no mismatch was found.

Part B: Test different expressions

Your program must handle the following examples:

Expression	Expected result
(A+B)	Balanced
((A+B)*C)	Balanced
{A+[B*(C+D)]}	Balanced
(A+B]	Not balanced
((A+B)	Not balanced
A+B)	Not balanced
{[()]}	Balanced
{[(])}	Not balanced

Part C: Display the reason for failure

If an expression is invalid, display a useful message such as:

- Closing bracket encountered without a matching opening bracket.
- Mismatched opening and closing brackets.
- One or more opening brackets remain unclosed.

Part D: Menu-driven application

Provide these options:

1. Check an expression
2. Validate multiple expressions
3. Exit

For option 2, allow the user to enter and validate five expressions in one run.

Additional challenges

1. Modify the program to detect an empty pair of parentheses such as ().
2. Report the position of the first mismatched bracket.
3. Extend the validator to check whether an expression contains valid operand and

operator ordering, such as rejecting A++B.
Note: The third challenge requires additional expression-state logic; balanced brackets alone do not guarantee a syntactically valid mathematical expression.

Question 4: Infix to Postfix Conversion and Postfix Evaluation
Difficulty: Intermediate to Advanced
Write a Java program that converts an infix mathematical expression into postfix notation and evaluates the resulting postfix expression using stacks implemented with arrays.
Do not use Java Collections Framework classes or third-party expression evaluation libraries.
Basic concepts
An infix expression places the operator between operands.
Example:
A + B
A postfix expression places the operator after its operands.
Example:
AB+
Postfix notation removes the need for parentheses to specify operator precedence during evaluation.

Requirements
Create two stack implementations:
1. CharStack – stores operators and parentheses during conversion.
2. IntStack – stores numeric operands during evaluation.
Both must use arrays and a top variable.

Part A: Convert infix to postfix
Implement:
Java
String infixToPostfix(String expression)
Your conversion algorithm must support:
- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Exponentiation (^)
- Parentheses
- Multi-digit integer operands

Use the following precedence rules:

Operator	Precedence	Associativity
^	3	Right to left
*, /	2	Left to right
+, -	1	Left to right

Use an operator-precedence method to decide when to push or pop operators.
Important: Exponentiation is right-associative. For example, 2^3^2 is conventionally interpreted as 2^(3^2), not (2^3)^2.

Part B: Evaluate postfix expressions
Implement:
Java

int evaluatePostfix(String postfix)
For each token:
1. If the token is an operand, push its numeric value.
2. If the token is a binary operator, pop the right operand first.
3. Pop the left operand second.
4. Apply the operator and push the result.
5. At the end, exactly one result should remain on the stack.

Part C: Menu-driven interface
========== EXPRESSION CALCULATOR ==========
1. Convert Infix to Postfix
2. Evaluate Postfix
3. Convert and Evaluate Infix
4. Exit

Sample test cases

Infix expression	Postfix expression
A+B*C	A B C * +
(A+B)*C	A B + C *
A+B-C	A B + C -
2+3*4	2 3 4 * +
(8+2)*3	8 2 + 3 *
2^3^2	2 3 2 ^ ^

For the numeric examples, display both the postfix expression and its evaluated result.

Error handling

- Reject mismatched parentheses.
- Detect division by zero.
- Detect insufficient operands during postfix evaluation.
- Reject malformed expressions.
- Detect stack overflow and underflow.

Additional challenge: Extend the evaluator to support floating-point values, negative numbers, and unary minus.

Question 5: Browser History and Undo/Redo Using Two Stacks
Difficulty: Advanced
Develop a Java program that simulates browser navigation and text-editor undo/redo operations using manually implemented array-based stacks.
This question tests stack operations, state management, multiple stacks, and application logic.

Part A: Browser history simulation
Create a class named BrowserHistory that uses two stacks:

- backStack – stores previously visited pages.
- forwardStack – stores pages that can be revisited using Forward.

Each stack must be implemented using an array.
Implement these methods:
Java
```
void visitPage(String url);
void goBack();
void goForward();
void displayCurrentPage();
```

void displayBackHistory();
void displayForwardHistory();

Rules

Visit a page

- If the user visits a new URL, push the current page onto the back stack if one exists.
- Set the new URL as the current page.
- Clear the forward stack because the navigation path has changed.

Go Back

- If the back stack is not empty, push the current page onto the forward stack.
- Pop the most recent page from the back stack and make it current.
- If the back stack is empty, display an appropriate message.

Go Forward

- If the forward stack is not empty, push the current page onto the back stack.
- Pop the next page from the forward stack and make it current.
- If the forward stack is empty, display an appropriate message.

Part B: Undo and redo operations

Implement a simple text editor that supports:

- Entering a new text state.
- Undoing the latest change.
- Redoing an undone change.
- Displaying the current text.
- Displaying the available undo and redo history.

Use two stacks:

- undoStack
- redoStack

Each stack should store previous text states using a manually implemented String[] array.

Undo logic

1. If the undo stack is not empty, push the current text onto the redo stack.
2. Pop the latest saved state from the undo stack.
3. Make the popped state the current text.

Redo logic

1. If the redo stack is not empty, push the current text onto the undo stack.
2. Pop the latest state from the redo stack.
3. Make it the current text.

When the user makes a new edit after undoing, clear the redo stack.

Part C: Main menu

```
================================
     STACK APPLICATION
================================
1. Browser History
2. Text Editor Undo/Redo
3. Exit
```

Inside each module, provide a separate menu for the supported operations.

Part D: Testing requirements

Your program must demonstrate the following browser navigation:

Visit: google.com
Visit: wikipedia.org
Visit: oracle.com
Go Back

Current Page: wikipedia.org
Go Back
Current Page: google.com
Go Forward
Current Page: wikipedia.org
Visit: mysql.com
Go Forward
No forward history available.
For the text editor, test the following sequence:
Enter text: Hello
Enter text: Hello Java
Enter text: Hello Java Programming
Undo
Current Text: Hello Java
Undo
Current Text: Hello
Redo
Current Text: Hello Java
Additional challenges:

- Set a maximum capacity for both stacks.
- Display a message when the history stack becomes full.
- Implement a method that displays the oldest and newest saved states.
- Prevent invalid undo or redo operations when no corresponding history exists.